

Level 4: EDA-Man

Introducción

En esta práctica vamos a implementar un juego basado en cuatro robots autónomos y un robot controlado por un jugador.

Start up

Para empezar, desempolva tu proyecto de la segunda práctica (EDAPark) y Pruébalo con la aplicación EDA-Man, que encontrarás junto a este enunciado.

Todo debería funcionar como antes, salvo por el entorno nuevo.

EDA-Man

EDA-Man es un juego de un jugador que consiste en capturar 244 *dots* (puntos pequeños) y 4 *energizers* (puntos grandes), dispuestos en un tablero de 28 por 36 teselas. Cada tesela mide 10 cm x 10 cm.

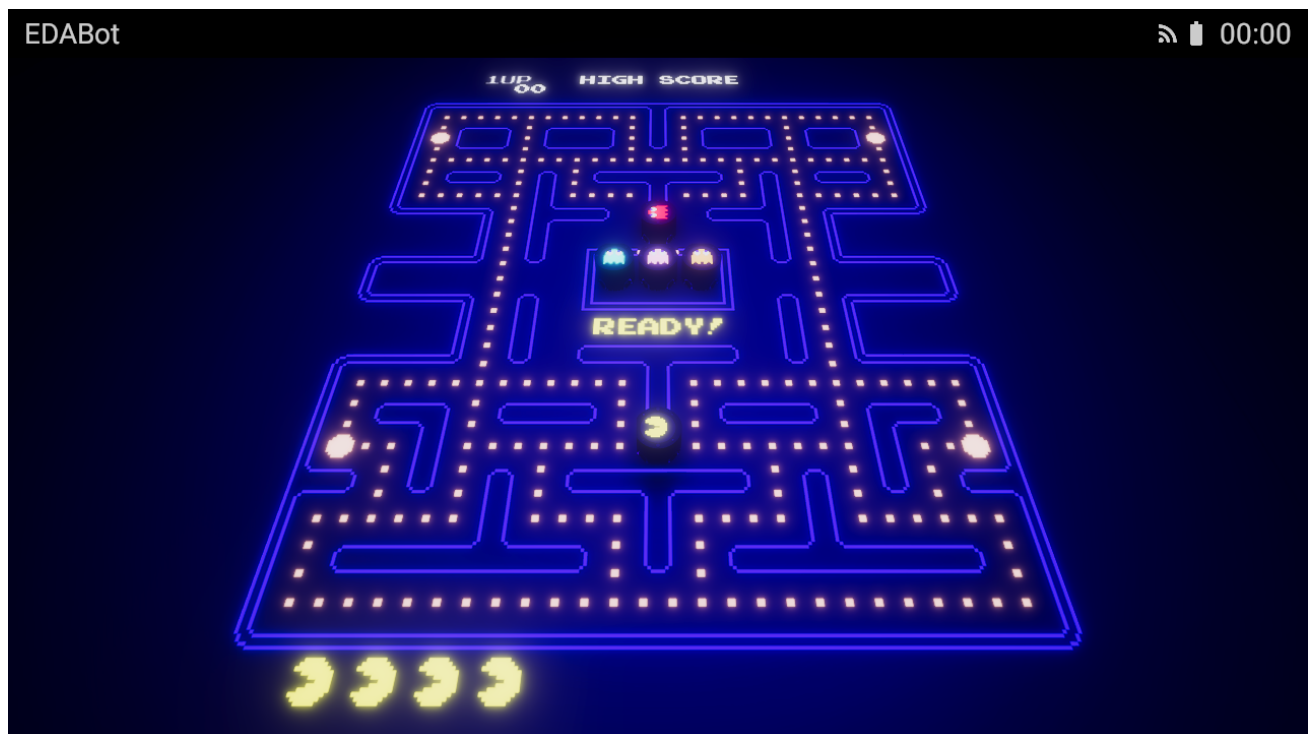


Fig. 1 - EDA-Man

El jugador es representado por un EDABot.

Hay cuatro enemigos, "Red", "Pink", "Cyan" y "Orange", también representados por EDABots, que tratan de atrapar al jugador.

El jugador puede moverse hacia arriba, abajo, a derecha o la izquierda en cualquier momento, siempre que no haya un obstáculo que lo bloquee. Los enemigos toman las decisiones de giro sólo cuando se topan con un cruce.

En el comienzo del juego, "Red" y "Pink" están libres. "Cyan" es liberado cuando el jugador captura 30 dots, y "Orange", cuando captura 60.

El jugador tiene cuatro vidas. Si el jugador se encuentra a menos de 10 cm de un enemigo, pierde una vida y el juego se re-inicia. El juego termina cuando se pierden todas las vidas.

Cuando el jugador captura un energizer, los enemigos se vuelven azules durante seis segundos. Entonces, si un enemigo se encuentra a menos de 10 cm del robot del jugador, es capturado: el juego se detiene brevemente y el enemigo es transportado con una grúa virtual a la guarida de enemigos.

Normalmente, el jugador se mueve a 0.64 m/s y los enemigos a 0.6 m/s. Cuando los enemigos están azules, el jugador se mueve a 0.72 m/s y los enemigos a 0.4 m/s.

Cuando los enemigos no están azules, alternan entre el modo *dispersión* y el modo *persecución* según la siguiente secuencia: 7 segundos de dispersión, 20 segundos de persecución, 7 segundos de dispersión, 20 segundos de persecución, 5 segundos de dispersión, 20 segundos de persecución, 5 segundos de dispersión, y luego persecución indefinida.

Cuando un enemigo está en modo dispersión o persecución, trata de alcanzar una tesela de destino; cuando está azul, deambula al azar.

Para alcanzar la tesela destino, en las intersecciones los enemigos seleccionan aquella dirección cuya tesela posee menor distancia a la tesela de destino (más detalles en [The Pacman Dossier](#), sección *Intersections*).

En modo dispersión, los enemigos tratan de alcanzar la tesela indicada en la figura 2.

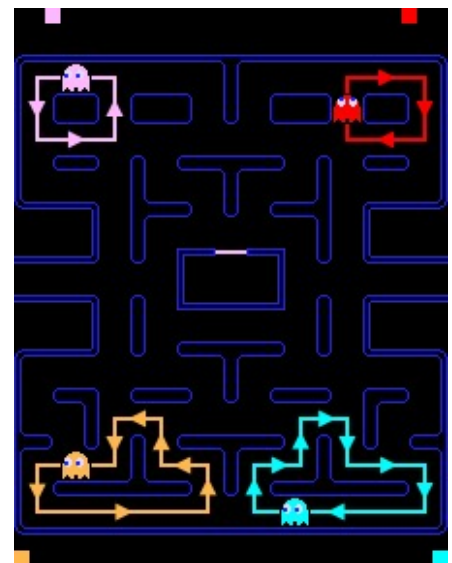


Fig. 2 - Modo dispersión

En modo persecución, los enemigos buscan alcanzar la siguiente tesela:

- “Red” intenta ir directamente a donde está el jugador (Fig. 3).
- “Pink” intenta ir a la tesela a 40 cm delante del jugador (Fig. 4).
- “Cyan” tiene un destino más complejo: respecto de la posición a 20 cm delante del jugador, proyecta el vector desde “Red” hasta ese punto, y lo extiende una vez más (Fig. 5).
- “Orange” es más sencillo: si se encuentra a una distancia de más de 80 cm del jugador, lo persigue al igual que “Red”. De lo contrario se comporta como en modo dispersión.

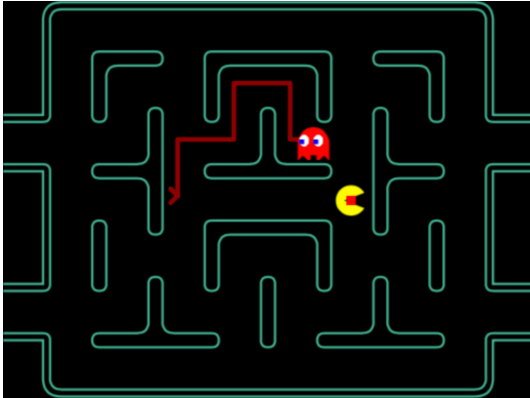


Fig. 3 - “Red”

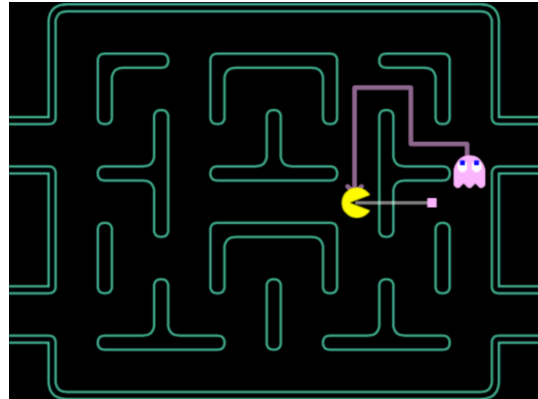


Fig. 4 - “Pink”

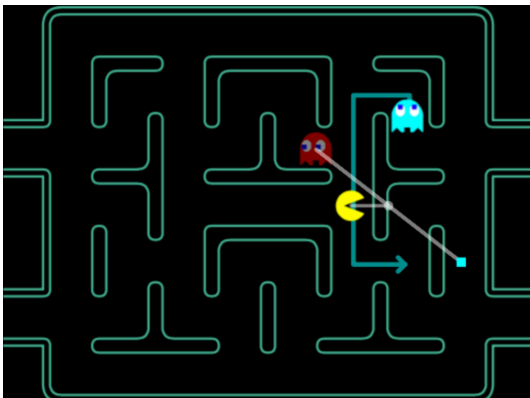


Fig. 5 - “Cyan”

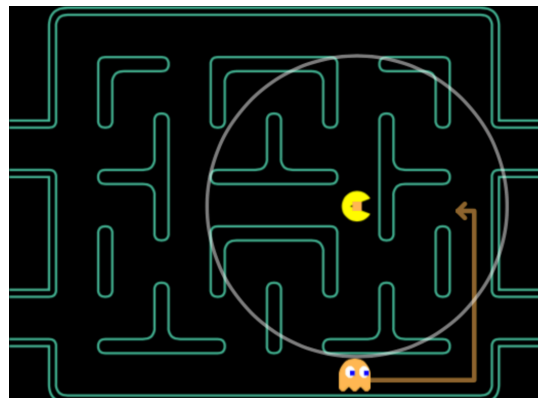


Fig. 6 - “Orange”

Para comprender esto mejor, ve a [este remake de Pac-Man](#), selecciona “Pac-Man”, y luego “Practice”.

Los dots valen 10 puntos, y los energizers, 50. El primer, segundo, tercer y cuarto enemigo capturado en fila vale 200, 400, 800 y 1600 puntos, respectivamente.

Hay muchos más detalles de EDA-Man que tú mismo puedes definir. No se trata de recrear el original, sino de crear una experiencia agradable al jugador.

A programar

Lo primero que debería ocuparte es el control de posición de los robots. Para ello te proveemos un controlador PID en cada robot (si quieres saber más sobre esto, consulta el anexo de este enunciado).

El controlador PID posee los siguientes topics MQTT:

Topic	Descripción	Payload	Acceso
[robotId]/pid/setpoint/set	Posición x, z [m] y rotación r [°]	float[3]	Write
[robotId]/pid/parameters/set	Parámetros P, I, D del controlador de posición y parámetros P, I, D del controlador de rotación (por defecto: 20, 0, 6, 0.1, 0, 0.005).	float[6]	Write

Un mensaje a **pid/setpoint/set** hará que el robot intente alcanzar la posición **x** (izquierda-derecha), **z** (adelante-atrás) y la rotación **r** (respecto del eje vertical, con 0° al norte) indicada en el mensaje.

Los robotId de los cinco robots son **robot1**, **robot2**, **robot3**, **robot4**, **robot5**.

Algunas recomendaciones:

- **Usa robot1 para el jugador.** Su cámara es accesible desde la aplicación EDA-Man.
- **Trabaja a “lazo abierto”.** No necesitas recibir datos del robot, puedes confiarte en que el controlador PID replicará el movimiento que determinas en tu código.
- **Envía continuamente setpoints a cada robot.** Esto te permite controlar su velocidad en forma simple.

Hook virtual

La aplicación EDA-Man dispone de un hook virtual (similar a los de las máquinas de gancho de peluches) que permite trasladar un robot a un destino (respecto del centro de la parte inferior del robot). Para activarlo, envía un mensaje MQTT con el siguiente formato:

Topic	Descripción	Payload	Acceso
hook/[robotId]/set	Posición x, y, z de destino [m]	float[3]	Write

La coordenada **x** es respecto de izquierda-derecha, la **y**, respecto de arriba-abajo, la **z**, respecto de adelante-atrás.

La duración del transporte depende de la distancia. Los mensajes del hook virtual no se encolan.

LED Floor

Para controlar el *LED floor* (el piso de EDA-Man), envía mensajes MQTT con el siguiente formato:

Topic	Descripción	Payload	Acceso
ledfloor/tiles/set	Imprime teselas a partir de (x, y), con color "palette".	char x char y char palette char data[...]	Write

x e y son las coordenadas de tesela (respecto de arriba a la izquierda) de la primera tesela a ser pintada, **palette** es un color que tiñe las teselas (sólo las teselas **0x00** a **0x7f**, ver Fig. 7), y **data**, un arreglo de bytes que se pinta desde la coordenada x, y a derecha, fila a fila. Cada byte en **data** representa una tesela a ser pintada según el mapa en la figura 8:



Fig. 7 - Paleta de colores



Fig. 8 - Mapa de teselas

Recharge area

Los robots disponen de un área de recarga de batería, que abarca un círculo de 20 cm de radio alrededor de la posición inicial de cada robot.

Jukebox

EDA-Man dispone de un *jukebox* para reproducir sonidos. Admite los siguientes mensajes MQTT:

Topic	Descripción	Payload	Acceso
jukebox/[audiold]/play	Reproduce el sonido [audiold]	-	Write
jukebox/[audiold]/stop	Detiene el sonido [audiold]	-	Write

Los sonidos disponibles son:

- **"backgroundSiren0"**, **"backgroundSiren1"**, **"backgroundSiren2"**, **"backgroundSiren3"**, **"backgroundSiren4"** (puedes usarlos como sonido de fondo, cambiando de sirena conforme el jugador va comiendo puntos), **"backgroundEnergizer"** (cuando los robots se vuelven azules), **"backgroundGhostsCaptured"** (mientras un robot capturado es transportado a la guarida).
- **"eatingDots"** (cuando se comen dots o energizers), **"eatingGhost"** (cuando se captura un enemigo), **"eatingFruit"** (opcional: cuando el jugador come una fruta).
- **"mainStart"** (cuando comienza el juego), **"mainLost"** (cuando el jugador pierde una vida), **"mainWon"** (cuando se gana el nivel), **"mainIntermission"** (opcional: entre niveles), **"mainInsertCoin"** (opcional: cuando se inserta un coin).

Starter code

Para que te resulte más fácil programar el proyecto, te proveemos un starter code con una clase **GameView** que implementa la vista del juego.

También te proveemos una clase **GameModel**, que puedes completar para escribir el modelo del juego.

Además te proveemos la clase base **Robot**, que puedes completar para implementar los comportamientos de cada robot.

Recomendaciones

- **¡Prioriza!** Este es un proyecto ambicioso, que da mucha satisfacción una vez terminado. Pero requiere mucha coordinación de equipo.
- **¡Prioriza!** Piensa en qué es esencial y qué es secundario. Deja lo secundario para el final.
- Frecuentemente deberás convertir entre coordenadas de mundo y coordenadas de teselas (del LED floor). Aprovecha las funciones que te damos en la clase **Robot**.

- **Utiliza programación orientada por eventos.** Añade métodos `on[Event]` para que tus clases derivadas se enteren de cierto evento. Por ejemplo, puedes definir `onCrossing()` para cuando un robot llega a una intersección.
- **Evita que se quemen los motores.** Puedes cambiar las velocidades de los robots (apartándote de lo especificado en el enunciando, siempre que justifiques), o ajustar los parámetros del PID (disminúyelos para un control más permisivo).

Bonus points

- Implementa giros suaves.
- Implementa frutas.
- Implementa el túnel del juego original (si quieres, puedes aprovechar el hook).
- Añade más niveles, con intermisiones entre niveles. Mejor aún, añade tus propios laberintos.
- Sube un buen vídeo de tu trabajo a YouTube o Vimeo que luego podrás añadir a tu curriculum vitae.

Anexo

Un *controlador de posición PID* (proporcional, integral y derivativo) es un mecanismo de control que permite regular la posición de un robot a través de un lazo de realimentación.

El siguiente esquema detalla su funcionamiento:

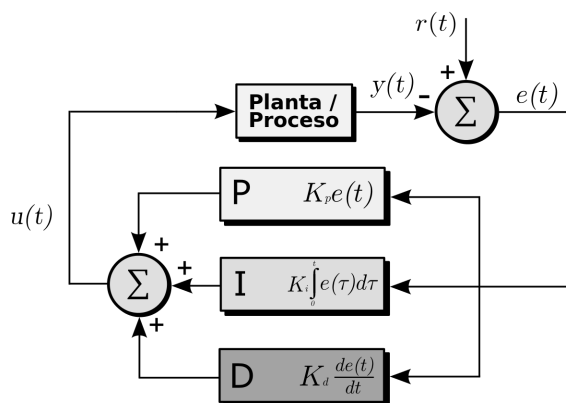


Fig. 9 - Esquema PID

El controlador de EDABot calcula continuamente el error entre la posición de la coordenada **x** (izquierda-derecha), de la coordenada **z** (adelante-atrás), la rotación **r** (respecto del eje vertical), y el **setpoint** que defines a través de MQTT.

La tensión de los motores se determina a partir de la suma del error multiplicado por la constante proporcional K_p , la integral del error multiplicado por la constante integral K_i y la derivada del error multiplicado por la constante derivativa K_d .

Cambiando los valores K_p , K_i y K_d puedes tunear el comportamiento del controlador.